

ASSIGNMENT 1 REPORT

Normal mode : <https://youtu.be/dzo2QkrtFEU>
Modified : <https://youtu.be/nP9QoJVIgZA>

INTRODUCTION

In this project, I developed a simple computer game similar to DX-BALL using Java and the StdDraw library. The player controls a paddle at the bottom of the screen, using it to bounce a ball and break bricks positioned at the top. When the ball strikes a brick, the brick is destroyed, and the ball rebounds. The objective is to clear all the bricks without letting the ball fall past the paddle.

The main features implemented in this game include ball launching, paddle movement, ball movement, collision detection and bouncing, pause functionality, victory and game-over states, a restart option (in the modified version), and rendering game elements.

GAME MECHANICS

SHOOTING : The initial angle is set to 90 degrees and can be adjusted using the Left and Right arrow keys. The angle ranges from a minimum of 0 degrees to a maximum of 180 degrees. The length of the arrow line remains fixed; however, its final x and y coordinates depend on the angle, calculated using cosine and sine functions. This allows the player to visualize the ball's initial trajectory.

The ball's speed is fixed, but its initial velocity components are determined based on the angle using cosine and sine functions. Before the shot, both the ball and paddle speeds are set to zero. When the Space key is pressed, the ball is launched, and their speeds are set to their actual values.

BALL MOVE : The ball's movement is determined by four components: its x and y positions (center coordinates) and its x and y velocity components. The next position is calculated by adding the velocity components to the current position.(as shown in equation 1) When collisions occur, the velocity components change accordingly.

$$(x1, y1) = (x0 + vx, y0 + vy) \quad (1)$$

PADDLE MOVE: The paddle has a fixed y-position but can move back and forth along the x-axis. The player controls its movement using the Left and Right arrow keys, which decrease or increase the paddle's x-position (center) based on the paddle's speed. (as shown in equation 2)

$$(x1, y1) = (x0 + vx, y0) \quad (2)$$

COLLISIONS: The ball can bounce off walls, the ceiling, bricks, and the paddle. When colliding with bricks or the paddle, it may undergo either a surface collision or a corner collision. The ball's movement follows the principles of elastic collisions.

SURFACE COLLISION: First, the surface collision condition is checked by comparing the distance between the object and the ball with the ball's radius. When the ball collides with walls, the ceiling, bricks, or the paddle on their surface, it bounces off at an angle equal to the angle of incidence. This is achieved by reversing the corresponding x or y velocity component while maintaining its magnitude. (as shown in Figure 1)

$$(vx1, vy1) = (vx0, -vy0) \text{ or } (vx1, vy1) = (-vx0, vy0) \quad (3)$$

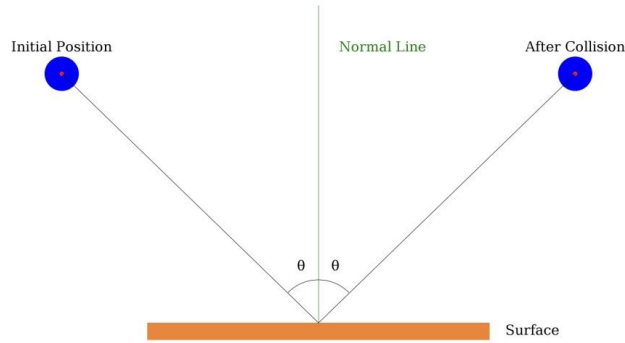


Figure 1: Surface collision

CORNER COLLISION: First, the corner collision condition is checked by comparing the distance between the object's corner and the ball with the ball's radius. Additionally, the direction of incidence is examined. If the incidence direction falls within the side region of the object rather than the cross region, the collision is considered a surface collision rather than a corner collision (as shown in Figure 2). This approach serves as an approximation. It avoids some of the bugs due to ball-brick overlapping.

Next, the closest point of the ball to the corner is determined. The normal line and the unit normal vector pointing from the corner to the ball's center are then calculated. (equation 4 and 5) Using these, the dot product of the initial velocity vector and the unit normal vector is computed. (equation 6, 7 and 8) The final velocity is determined using the formula in the equation 9 and 10. Its visual representation is given in Figure 3.

$$\text{NormalX} = \text{ball position } y - \text{closest } y \quad (4)$$

$$\text{NormalY} = \text{ball position } x - \text{closest } x \quad (5)$$

$$\text{DotproductX} = \text{ball velocity x} * \text{NormalX} \quad (6)$$

$$\text{DotproductY} = \text{ball velocity y} * \text{NormalY} \quad (7)$$

$$\text{DotProductTotal} = \text{DotproductX} + \text{DotproductY} \quad (8)$$

$$\text{Vxfinal} = \text{Vxinitial} - 2 * \text{DotproductTotal} * \text{NormalX} \quad (9)$$

$$\text{Vyfinal} = \text{Vyinitial} - 2 * \text{DotproductTotal} * \text{NormalY} \quad (10)$$

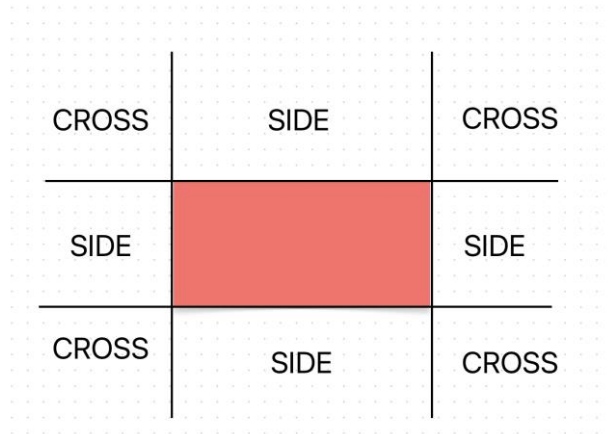


Figure 2 : Cross and side regions

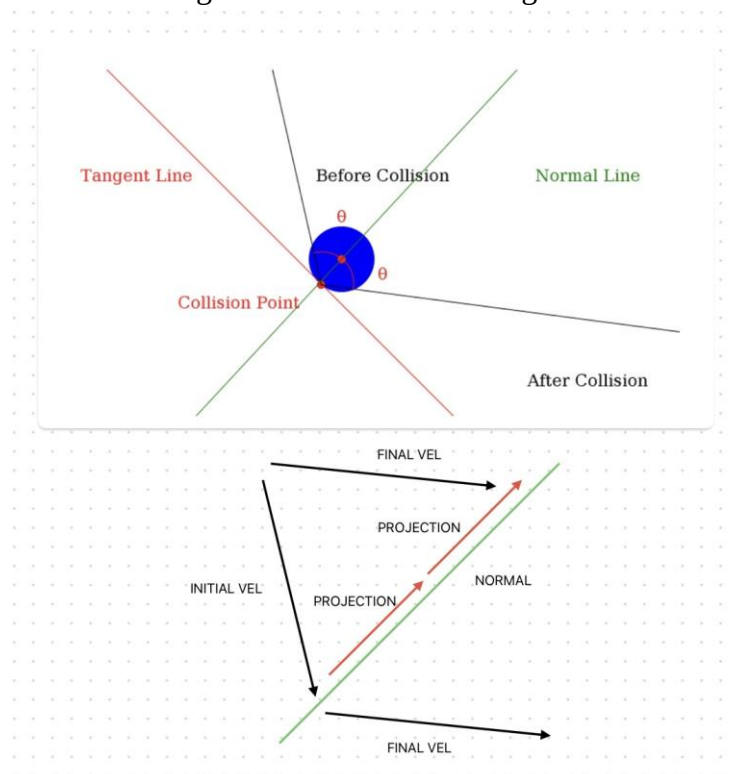


Figure 3 : Corner collision

IMPLEMENTATION DETAILS

```
// fonts
Font fontScore = new Font( name: "Serif", Font.BOLD, size: 30);
Font fontGameOver = new Font( name: "Serif", Font.BOLD, size: 50);
Font fontAngle = new Font( name: "Serif", Font.BOLD, size: 20);
Font fontPaused = new Font( name: "Serif", Font.BOLD, size: 20);
```

Figure 4 : Fonts

Specific font types for different texts are defined in Figure 4.

```
int brickNumber = brickColors.length;
// Brick existence value
int[] brickCheck = new int[brickNumber];
for (int i = 0; i < brickNumber; i++) {
    brickCheck[i] = 1;
}
```

Figure 5 : Brick existence check

In Figure 5, an array is used to track the existence of each brick, allowing the game to check whether the ball has collided with a brick.

```

// initial values
double ballPositionX = initialBallPos[0];
double ballPositionY = initialBallPos[1];
double paddlePositionX = paddlePos[0];
double paddlePositionY = paddlePos[1];
int count = 0;
boolean isShooted = false;
boolean isGameOver = false;
boolean isVictory = false ;
boolean isPaused = false;
double angle = initialAngle;
double ballVelocityX = ballVelocity * Math.cos(Math.toRadians(angle));
double ballVelocityY = ballVelocity * Math.sin(Math.toRadians(angle));
double ballVelocityXBeforePause = 0 ;
double ballVelocityYBeforePause = 0 ;
double paddleSpeed = initialPaddleSpeed;
double collidedBricksNumber = 0;
boolean keyPressed = true;

boolean isCornerCollision = false;
double[] cornersX = new double[2];
double[] cornersY = new double[2];
double distance ;
double closestX = 0 ;
double closestY = 0 ;
double normalX ;
double normalY ;
double dotProductX;
double dotProductY;
double dotProductTotal;
double length = 1 ;

```

Figure 6 : Variables

In Figure 6, all the variables used in the while loop are declared just before the loop starts and are set to their initial values if necessary.

```

// main loop
while (true) {
    // BEFORE SHOOT
    while (!(isShooted)){
        ballVelocity = 0 ;
        // shooting angle change
        if (StdDraw.isKeyPressed(KeyEvent.VK_RIGHT)){
            if ( angle > 0 ){
                angle = angle - 1 ;
            }
        }
        if (StdDraw.isKeyPressed(KeyEvent.VK_LEFT)){
            if ( angle < 180 ){
                angle = angle + 1;
            }
        }
        // shoot with space
        if (StdDraw.isKeyPressed(KeyEvent.VK_SPACE)) {
            ballVelocity = 5 ;
            ballVelocityX = ballVelocity * Math.cos(Math.toRadians(angle));
            ballVelocityY = ballVelocity * Math.sin(Math.toRadians(angle));
            isShooted = true;
            break;
        }
    }
}

```

Figure 7 : Before Shoot

In Figure 7, the main while loop begins. It consists of two main parts. The first part is the "Before Shoot" section, where the ball has not been shot yet. The player can adjust the shooting angle using the right and left arrow keys. The angle can range from a minimum of 0 to a maximum of 180 degrees, and the current angle is displayed in the top-left corner. When the 'SPACE' key is pressed, the ball starts moving at a speed of 5. The 'break' statement terminates the "Before Shoot" while loop.

```
// DRAW
StdDraw.clear(StdDraw.WHITE);
// draw ball on the screen
StdDraw.setPenColor(ballColor);
StdDraw.filledCircle(ballPositionX, ballPositionY, ballRadius);
// draw paddle on the screen
StdDraw.setPenColor(paddleColor);
StdDraw.filledRectangle(paddlePositionX, paddlePositionY, paddleHalfwidth, paddleHalfheight);

// draw angle on left corner
StdDraw.setPenColor(StdDraw.RED);
StdDraw.setFont(fontAngle);
String angle_text = String.format("ANGLE = %3.1f", angle);
StdDraw.text(x: 70, y: 350, angle_text);

// draw shooting arrow
StdDraw.setPenColor(StdDraw.RED);
StdDraw.setPenRadius(0.01);
StdDraw.line(ballPositionX, ballPositionY, x1: ballPositionX + arrowLenght * Math.cos(Math.toRadians(angle)),
             y1: ballPositionY + arrowLenght * Math.sin(Math.toRadians(angle)));

// draw bricks
for (int i = 0 ; i < brickNumber ; i++){
    if (brickCheck[i] ==1){
        StdDraw.setPenColor(brickColors[i]);
        StdDraw.filledRectangle(brickCoordinates[i][0], brickCoordinates[i][1], brickHalfwidth, brickHalfheight);
    }
}

StdDraw.show();
StdDraw.pause(pauseDuration);
```

Figure 8 : Rendering before shoot

In Figure 8, the game components for the "Before Shoot" phase (paddle, ball, bricks, shooting arrow, and angle text) are drawn. Figure 9 shows how these components appear during gameplay.

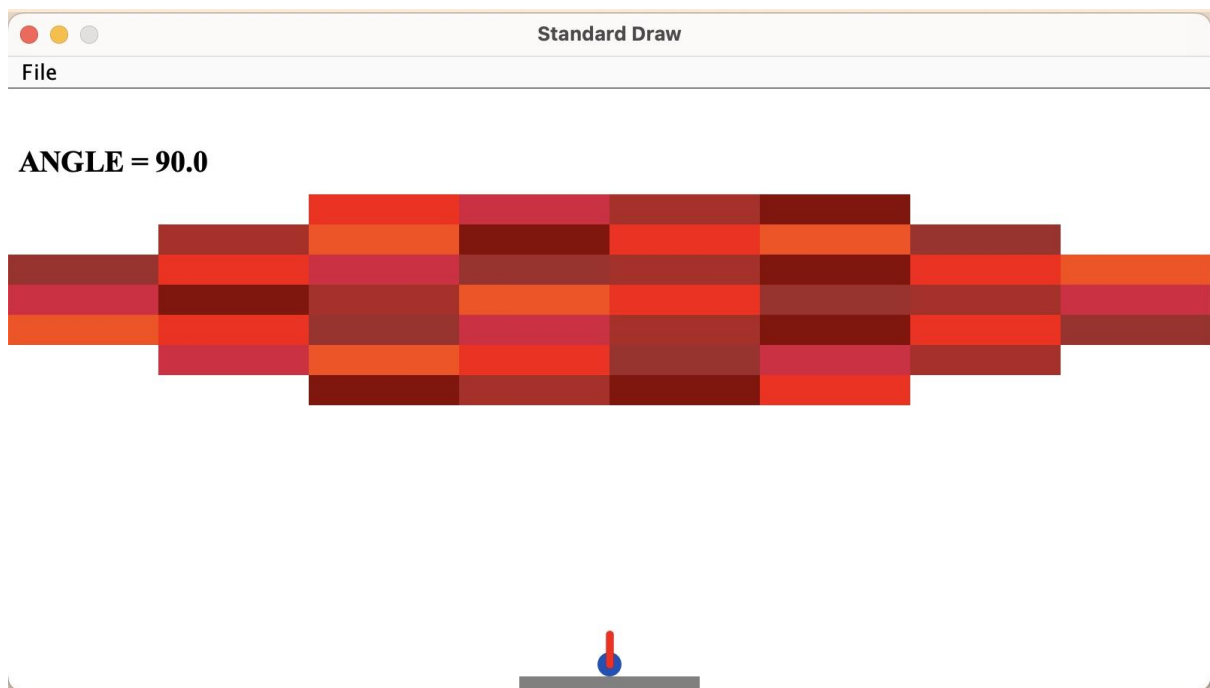


Figure 9 : Before shoot look

```
// AFTER SHOOT
// bounce off wall and gameover check
if ((ballPositionX + ballVelocityX) > canvas_width - ballRadius || (ballPositionX + ballVelocityX) < 0 + ballRadius)
    ballVelocityX = -ballVelocityX;
if ((ballPositionY + ballVelocityY) > canvas_height - ballRadius)
    ballVelocityY = -ballVelocityY;
if ((ballPositionY + ballVelocityY) < 0 + ballRadius){
    ballVelocityY = 0;
    ballVelocityX = 0;
    paddleSpeed = 0;
    isGameOver = true;
}
```

Figure 10 : Bounce off wall

In Figure 10, the "After Shoot" code segment begins. It consists of four parts: collision checks and bounces, pause check, paddle movement and ball movement, and drawing the game elements. The collision check starts with wall collisions. The ball bounces off the left, right, and top walls, but if it touches the bottom wall, the game is over.

```
// bounce off paddle
cornersX[0] = paddlePositionX - paddleHalfwidth;
cornersX[1] = paddlePositionX + paddleHalfwidth;
cornersY[0] = paddlePositionY - paddleHalfheight;
cornersY[1] = paddlePositionY + paddleHalfheight;
for (double cx : cornersX) {
    for (double cy : cornersY) {
        distance = Math.sqrt((ballPositionX - cx) * (ballPositionX - cx) + (ballPositionY - cy) * (ballPositionY - cy));
        if (distance <= ballRadius) {
            if ((ballPositionX < paddlePositionX - paddleHalfwidth) || (ballPositionX > paddlePositionX + paddleHalfwidth)) {
                isCornerCollision = true;
                closestX = cx;
                closestY = cy;
            }
        }
    }
}
```

Figure 11: Paddle corner collision check

In paddle and brick collisions, there are two primary scenarios: surface collision and corner collision. Figure 11 illustrates the positions of the paddle's corners, which are used in a for loop to detect whether a corner collision occurs between the ball and the paddle. If a corner collision is detected, the variable `isCornerCollision` is set to true, and the coordinates of the colliding corner are labeled as the "closest x and y" for calculating the unit normal vector.


```

if (paddlePositionY + paddleHalfheight + ballRadius > ballPositionY) {
    if ((ballPositionX > paddlePositionX - paddleHalfwidth) & (ballPositionX < paddlePositionX + paddleHalfwidth)) {
        ballVelocityY = -ballVelocityY;
    }
}

if (isCornerCollision){
    normalX = (ballPositionX - closestX) ;
    normalY = (ballPositionY - closestY) ;
    length = Math.sqrt(normalX * normalX + normalY * normalY);
    normalX = normalX / length;
    normalY = normalY / length ;
    dotProductX = (ballVelocityX * normalX) ;
    dotProductY = (ballVelocityY * normalY);
    dotProductTotal = dotProductX + dotProductY;
    ballVelocityX = (ballVelocityX - 2 * dotProductTotal * normalX );
    ballVelocityY = (ballVelocityY - 2 * dotProductTotal * normalY);
}
isCornerCollision = false;

```

Figure 12: Paddle bounce off

In Figure 12, the vertical surface collision between the ball and the paddle is checked first. The horizontal collision is ignored, as it is not relevant to the mechanics of this game. If a vertical surface collision is detected, the ball's Y velocity is updated accordingly.

In the second part of the process, a corner collision is evaluated, as described in the code block in Figure 11. If a corner collision is detected, the formula outlined in the game mechanics section is applied to handle it.

```

// bounce off bricks
for (int i = 0 ; i < brickNumber ; i++){
    if (brickCheck[i] == 1){
        // check corner collision
        cornersX[0] = brickCoordinates[i][0]- brickHalfwidth;
        cornersX[1] = brickCoordinates[i][0]+ brickHalfwidth;
        cornersY[0] = brickCoordinates[i][1]- brickHalfheight;
        cornersY[1] = brickCoordinates[i][1]+ brickHalfheight;

        for (double cx2 : cornersX) {
            for (double cy2 : cornersY) {
                distance = Math.sqrt((ballPositionX- cx2) * (ballPositionX - cx2) + (ballPositionY - cy2) * (ballPositionY - cy2));
                if (distance <= ballRadius) {
                    if ((ballPositionX > cornersX[1] || ballPositionX < cornersX[0]) && (ballPositionY > cornersY[1] || ballPositionY < cornersY[0])) {
                        isCornerCollision = true;
                        closestX = cx2;
                        closestY = cy2;
                    }
                }
            }
        }
    }
}

```

Figure 13 : Paddle corner collision check

In Figure 13, a similar process to Figure 11 is applied to check the corner collision between the bricks and the ball.

```

if (isCornerCollision){
    // calculate the normal line
    brickCheck[i] = 0;
    score += 10;
    count += 1;
    collidedBricksNumber += 1;
    if (collidedBricksNumber == 1) {
        normalX = (ballPositionX - closestX) ;
        normalY = (ballPositionY - closestY) ;
        length = Math.sqrt(normalX * normalX + normalY * normalY);
        normalX = normalX / length;
        normalY = normalY / length ;
        dotProductX = (ballVelocityX * normalX) ;
        dotProductY = (ballVelocityY * normalY);
        dotProductTotal = dotProductX + dotProductY;
        ballVelocityX = (ballVelocityX - 2 * dotProductTotal * normalX );
        ballVelocityY = (ballVelocityY - 2 * dotProductTotal * normalY);
    }
    if (count == brickNumber) {
        ballVelocityY = 0;
        ballVelocityX = 0;
        paddleSpeed = 0;
        isVictory = true;
        keyPressed = true;
    }
    isCornerCollision = false;
}

```

Figure 14 : Brick corner collision

In Figure 14, a similar process to Figure 12 is applied to handle the corner collision. The existence value of the touched brick is set to 0, meaning it will no longer be drawn or trigger a bounce. The score is updated, along with the total count of bricks hit by the ball and the number of bricks collided with in a single ball iteration. The latter ensures that the ball bounces only once during a single move (this parameter addresses the issue of a 90-degree shot). Additionally, the game checks if the victory condition has been met (i.e, if the ball has hit all the bricks).

```
// horizontal collision
if ((brickCoordinates[i][0] + brickHalfwidth >= ballPositionX - ballRadius) & (brickCoordinates[i][0] - brickHalfwidth <= ballPositionX + ballRadius))
    if ((ballPositionY >= brickCoordinates[i][1] - brickHalfheight) & (ballPositionY <= brickCoordinates[i][1] + brickHalfheight)) {
        brickCheck[i] = 0;
        score += 10;
        count += 1;
        collidedBricksNumber += 1;
        if (collidedBricksNumber == 1) {
            ballVelocityX = -ballVelocityX;
        }
        if (count == brickNumber) {
            ballVelocityY = 0;
            ballVelocityX = 0;
            paddleSpeed = 0;
            isVictory = true;
            keyPressed = true;
        }
    }
}
```

Figure 15 : Brick horizontal collision

```
// vertical collision
if ((brickCoordinates[i][1] + brickHalfheight >= ballPositionY - ballRadius) & (brickCoordinates[i][1] - brickHalfheight <= ballPositionY + ballRadius))
    if ((ballPositionX >= brickCoordinates[i][0] - brickHalfwidth) & (ballPositionX <= brickCoordinates[i][0] + brickHalfwidth)) {
        brickCheck[i] = 0;
        score += 10;
        count += 1;
        collidedBricksNumber += 1;
        if (collidedBricksNumber == 1) {
            ballVelocityY = -ballVelocityY;
        }
        if (count == brickNumber) {
            ballVelocityY = 0;
            ballVelocityX = 0;
            paddleSpeed = 0;
            isVictory = true;
            keyPressed = true;
        }
    }
}
```

Figure 16 : Brick vertical collision

In Figures 15 and 16, horizontal and vertical surface collisions are checked and handled. A process similar to the brick corner collision is followed for these checks.

```

if (!(isGameOver) && !(isVictory)) {
    if (!(isPaused)) {
        if (StdDraw.isKeyPressed(KeyEvent.VK_SPACE)) {
            if (!keyPressed) {
                isPaused = true;
                ballVelocityXBeforePause = ballVelocityX;
                ballVelocityYBeforePause = ballVelocityY;
                ballVelocityY = 0;
                ballVelocityX = 0;
                paddleSpeed = 0;
                keyPressed = true;
            } else {
                keyPressed = false;
            }
            StdDraw.pause(t: pauseDuration + 200);
        }
    } else {
        if (StdDraw.isKeyPressed(KeyEvent.VK_SPACE)) {
            if (keyPressed) {
                isPaused = false;
                ballVelocityY = ballVelocityYBeforePause;
                ballVelocityX = ballVelocityXBeforePause;
                paddleSpeed = initialPaddleSpeed;
                keyPressed = false;
            } else {
                keyPressed = true;
            }
            StdDraw.pause(t: pauseDuration + 200);
        }
    }
}
}

```

Figure 17 : Pause

In Figure 17, the pause feature is implemented. The KeyPressed status is checked to prevent multiple counts of the Space key. The ball's current velocity components are saved to be used after unpausing. To illustrate the pause, the ball and paddle velocities are set to 0, similar to the "Before Shoot" status. If the Space key is pressed while paused, it unpauses the game and restores the ball's velocity to its state just before the pause.

```

// paddle move
if (StdDraw.isKeyPressed(KeyEvent.VK_LEFT)) {
    if (paddlePositionX - paddleHalfwidth > 0) {
        paddlePositionX = paddlePositionX - paddleSpeed;
    }
}
if (StdDraw.isKeyPressed(KeyEvent.VK_RIGHT)) {
    if (paddlePositionX + paddleHalfwidth < canvas_width ) {
        paddlePositionX = paddlePositionX + paddleSpeed;
    }
}

// update ball position
ballPositionX = ballPositionX + ballVelocityX;
ballPositionY = ballPositionY + ballVelocityY;

```

Figure 18 : Paddle and ball move

In Figure 18, paddle movement and ball position updates are handled as described in the game mechanics.

```

// RESTART
if (isVictory || isGameOver){
    if (StdDraw.isKeyPressed(KeyEvent.VK_SPACE)){
        for (int i = 0 ; i < brickNumber; i++) {
            brickCheck[i] = 1;
            StdDraw.pause(keyPauseDuration);
        }
        isGameOver = false;
        isVictory = false;
        isShooted = false;
        keyPressed = true;
        score = 0;
        count = 0;
        angle = initialAngle;
        paddleSpeed = initialPaddleSpeed;
        ballPositionX = initialBallPos[0];
        ballPositionY = initialBallPos[1];
        paddlePositionX = paddlePos[0];
    }
}

```

Figure 19 : Restart (in modified version)

In the modified version, a restart feature is added, as shown in Figure 19. This feature allows the player to restart the game by pressing the Space key, either when the game is over or when the game is won. It essentially resets all the changing game variables to their initial values.

```

//DRAW
// clear
StdDraw.clear(StdDraw.WHITE);
// draw ball
StdDraw.setPenColor(ballColor);
StdDraw.filledCircle(ballPositionX, ballPositionY, ballRadius);
// draw paddle
StdDraw.setPenColor(paddleColor);
StdDraw.filledRectangle(paddlePositionX, paddlePositionY, paddleHalfwidth, paddleHalfheight);
// draw bricks
for (int i = 0 ; i < brickNumber ; i++){
    if (brickCheck[i] ==1){
        StdDraw.setPenColor(brickColors[i]);
        StdDraw.filledRectangle(brickCoordinates[i][0], brickCoordinates[i][1], brickHalfwidth, brickHalfheight);
    }
}
// draw pause
if (isPaused){
    StdDraw.setPenColor(StdDraw.BLACK);
    StdDraw.setFont(fontPaused);
    StdDraw.text(x: 70, y: 350, text: "PAUSED");
}

```

Figure 20 : Rendering

```

// draw game over
if (isGameOver) {
    StdDraw.setPenColor(StdDraw.BLACK);
    StdDraw.setFont(fontGameOver);
    StdDraw.text(x: xScale / 2, y: yScale / 2 - 100, text: "GAME OVER");
    StdDraw.setFont(fontScore);
    String score_text = String.format("Score : %d", score);
    StdDraw.text(x: xScale / 2, y: yScale / 2 - 150, score_text);
}
// draw victory
if (isVictory) {
    StdDraw.setPenColor(StdDraw.BLACK);
    StdDraw.setFont(fontGameOver);
    StdDraw.text(x: xScale / 2, y: yScale / 2 - 100, text: "VICTORY");
    StdDraw.setFont(fontScore);
    String score_text = String.format("Score : %d", score);
    StdDraw.text(x: xScale / 2, y: yScale / 2 - 150, score_text);
}
// draw score
if (!(isGameOver) && !(isVictory)){
    StdDraw.setPenColor(StdDraw.BLACK);
    StdDraw.setFont(fontScore);
    String score_text = String.format("Score : %d", score);
    StdDraw.text(x: xScale - 100 , y: yScale - 50 , score_text );
}
// show
StdDraw.show(); // show the drawing on the screen
StdDraw.pause(pauseDuration); // pause the drawing at each iteration

```

Figure 21 : Rendering 2

The code segments in Figures 20 and 21 draw the objects and texts on the screen, including the ball, paddle, remaining bricks, and the texts "PAUSED," "Score," "VICTORY,"

and "GAME OVER." Figures 22, 23, and 24 show how these elements appear during gameplay. In the modified version, the coordinates and colors of the bricks, the ball color, paddle color, and paddle width are changed. Figure 25 shows how the modified game looks during gameplay.

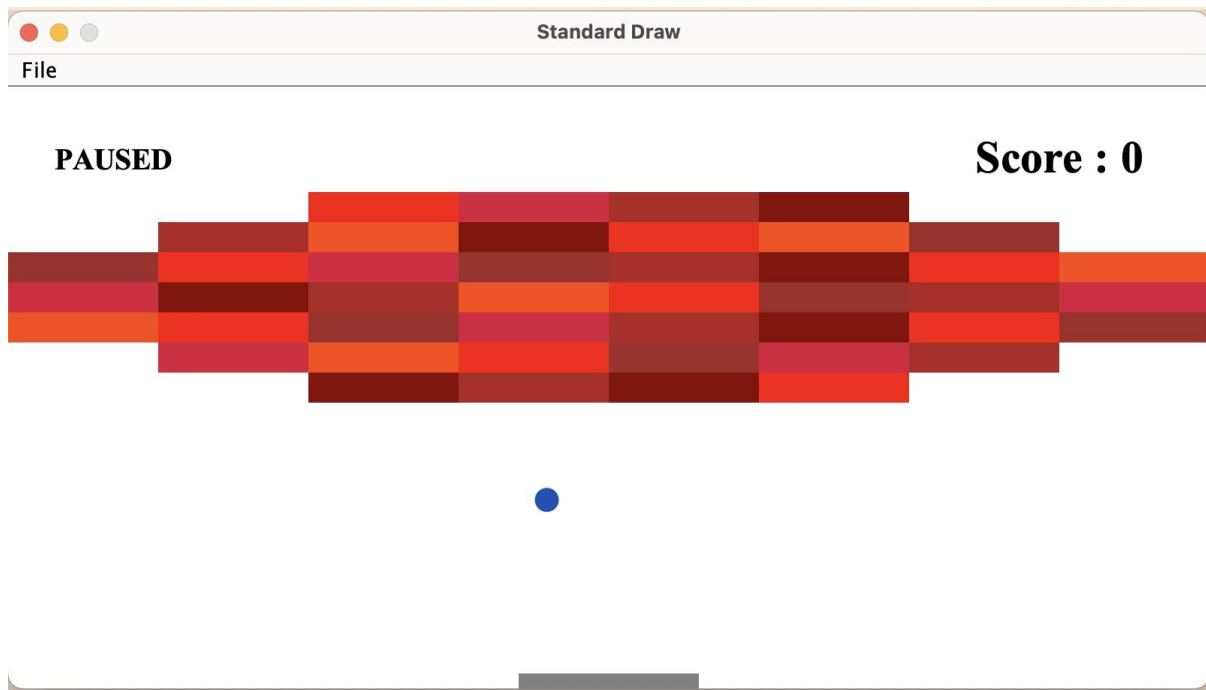


Figure 22 : Paused display

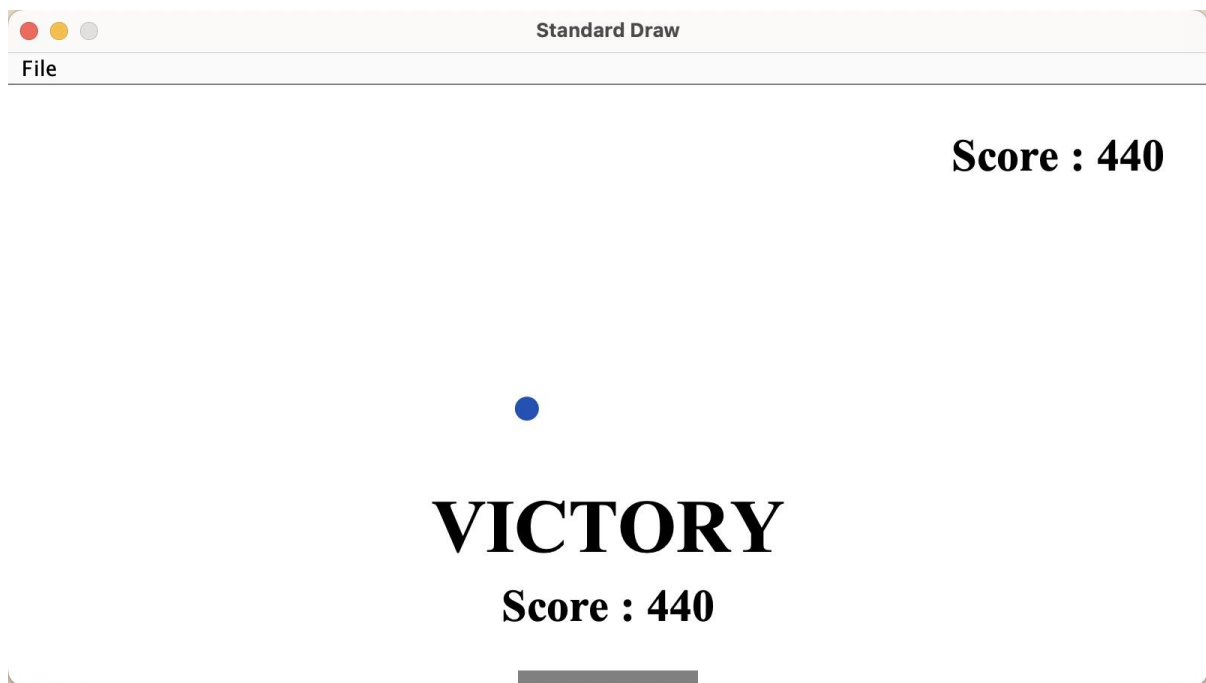


Figure 23 : Victory display

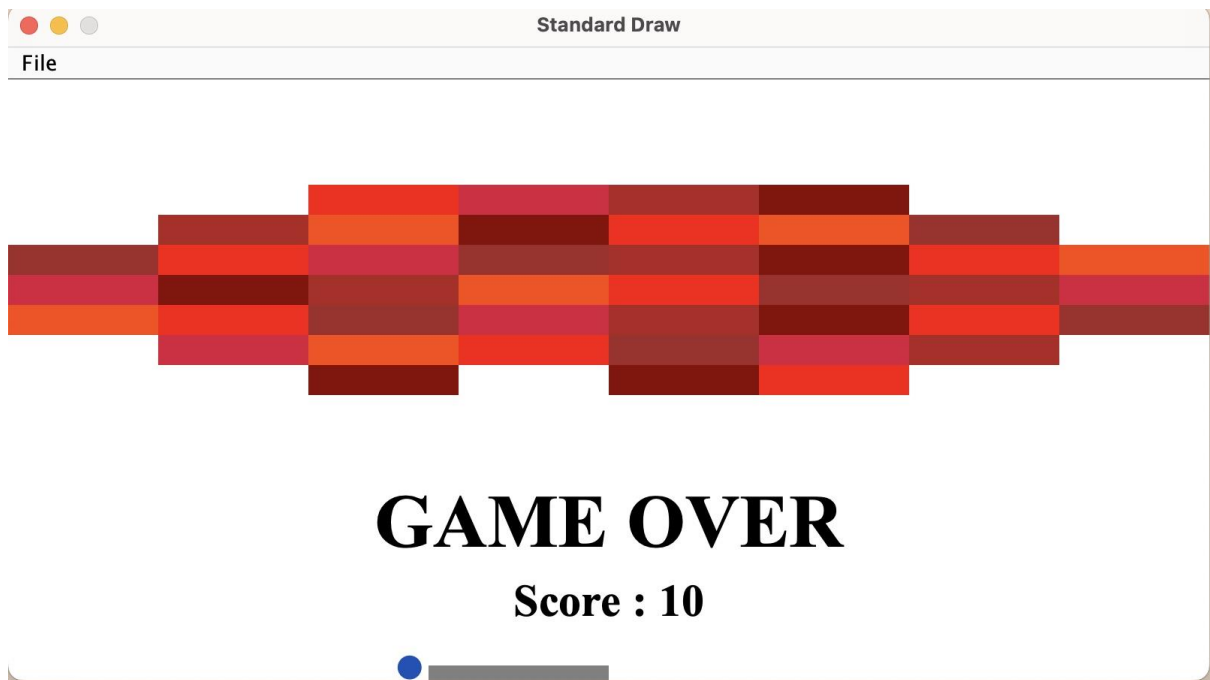


Figure 24 : Game over display

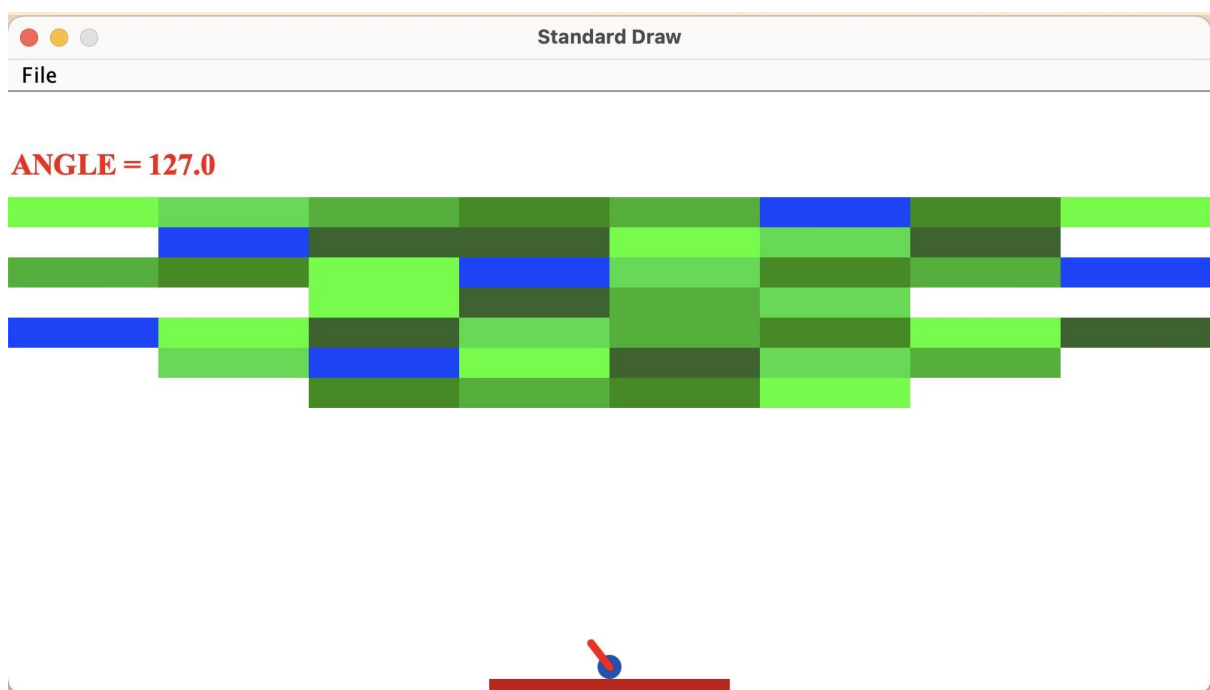


Figure 25 : Modified version display